

PROJECT

Raillo

기차 승차권 예매 서비스

코레일 예매 시스템을 모델로 구현한 기차 승차권 예매 서비스입니다. 열차 좌석은 단순 점유 자원이 아니라 이용 구간에 따라 재사용되며, 동일 좌석이라도 서울→대전, 대전→부산처럼 구간이 겹치지 않으면 서로 다른 승객이 동시에 예매할 수 있습니다.

이로 인해 좌석 예약은 구간 겹침 기반 충돌 판정 문제로 정의되며, 좌석 선점부터 결제까지의 과정에서 정합성을 보장하는 설계가 핵심 과제였습니다. 이를 위해 임시 점유와 확정 예매 흐름을 일관되게 관리하고, 동시성 제어와 조회 성능을 함께 고려한 예약 시스템을 설계했습니다.

프로젝트 정보

팀 구성 백엔드 3인

개발 기간 2025.06 ~ 2025.07, 2025.11~2026.03

담당 역할 열차·예약·결제 도메인 설계, 쿼리 및 성능 최적화, 인프라 구성

Raillo <https://github.com/penggu-dev/raillo-backend>

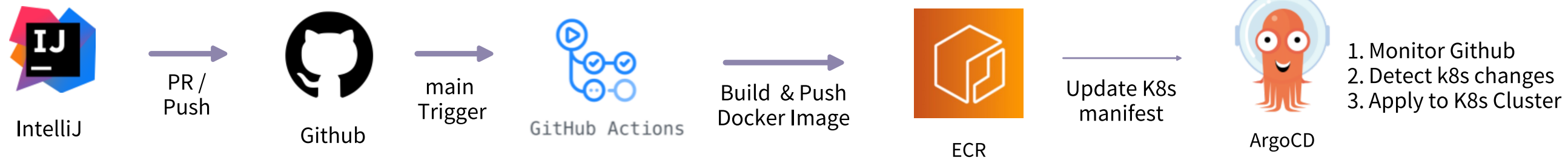
Batch <https://github.com/penggu-dev/raillo-batch>

기술 스택

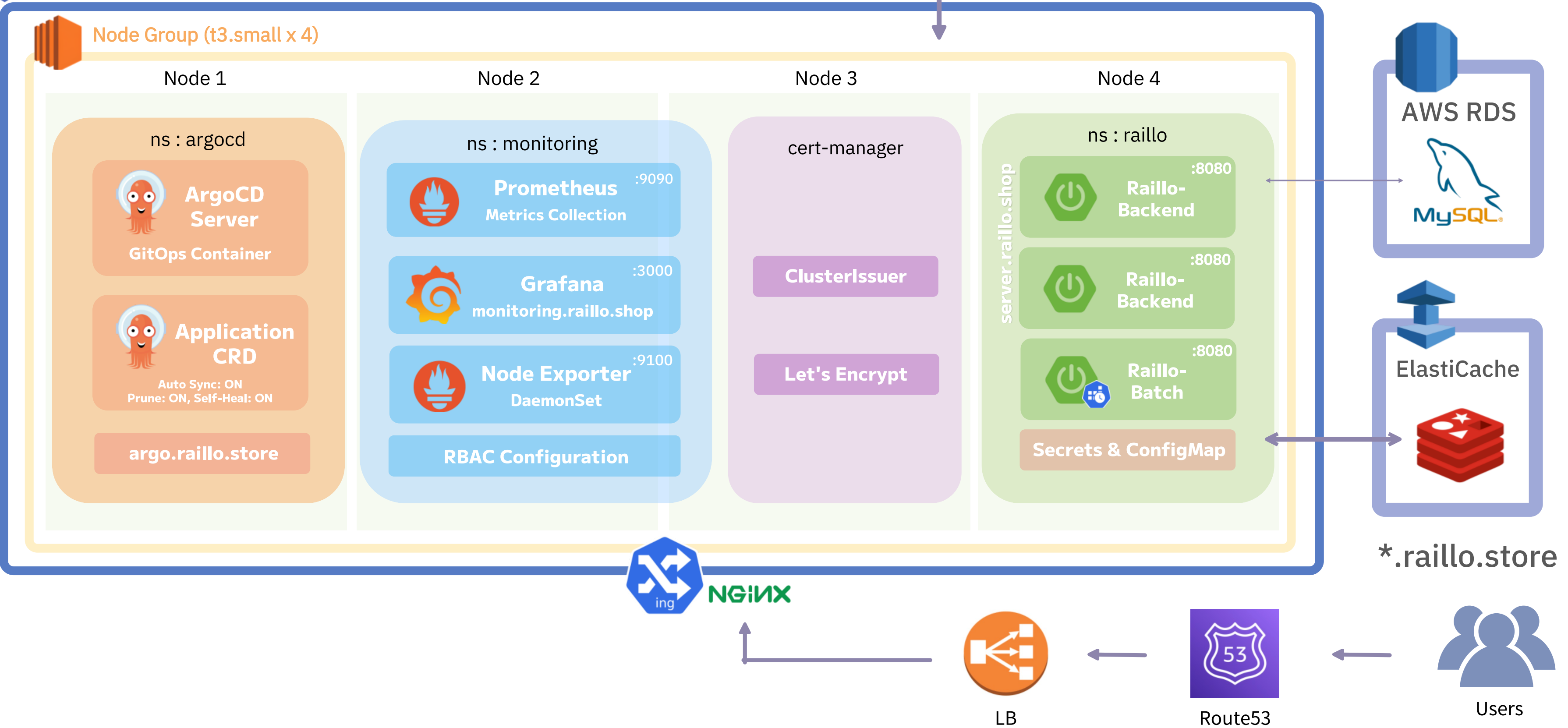
Backend Java 17, Spring Boot 3.5, JPA, QueryDSL, Redis, MySQL

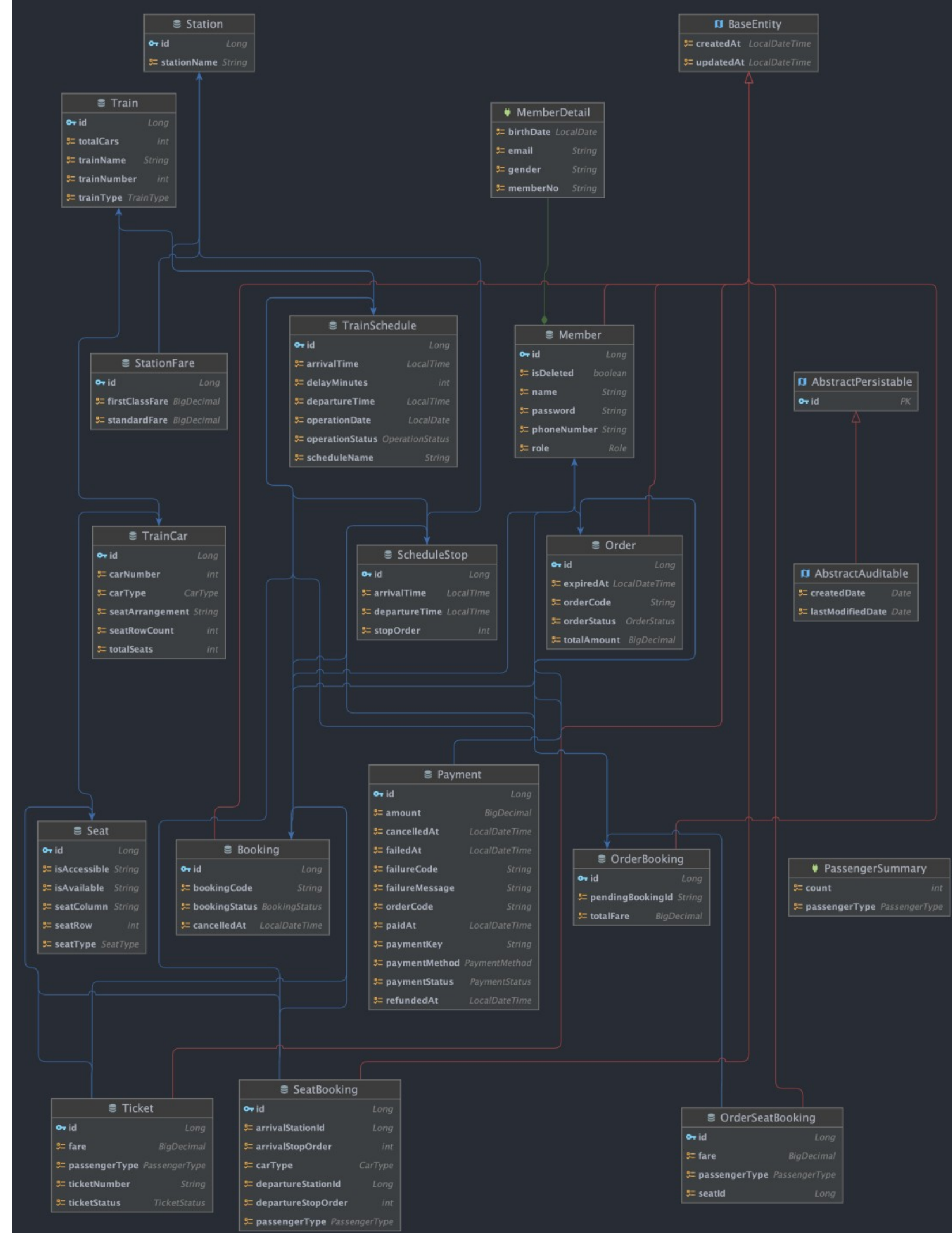
Infra AWS EKS, ElastiCache, RDS, Route53, ArgoCD, Github Actions

Monitoring Prometheus, Grafana, Spring Actuator



Raillo EKS





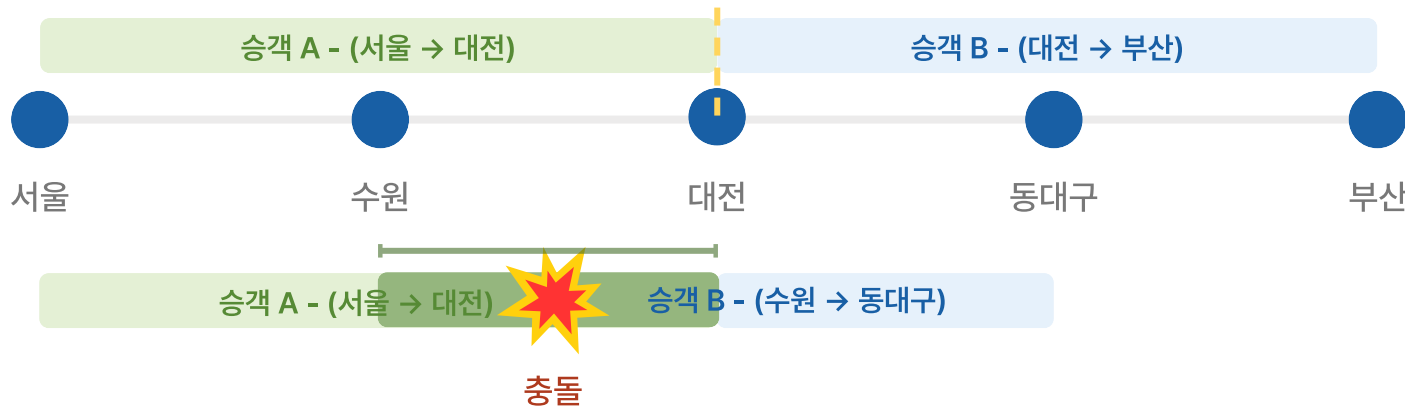
상세 ERD URL:
<https://velog.velcdn.com/images/ogu1208/post/66d3c022-f282-4185-9f2b-46597c21cc87/image.png>

철도 도메인 분석 & 엔티티 설계

구간 기반 좌석 점유란?

같은 좌석이라도 이용 구간이 겹치지 않으면 서로 다른 승객이 예약할 수 있어야 합니다. 단순 좌석 점유가 아닌 구간 단위 충돌 판단이 필요합니다.

예: 42번 좌석을 승객 A가 서울→대전 예약 시, 승객 B는 대전→부산 구간을 동일 좌석으로 예약 가능합니다. 반면, 서울→대전으로 예약된 상태에서 수원→동대구처럼 구간이 일부라도 겹치는 경우에는 동일 좌석을 예약할 수 없습니다.



열차 엔티티 구성

Train (열차)

차량 편성 등 열차 자체의 물리적 정보

TrainCar (객차)

일반실·특실 등 등급 구분 단위

Seat (좌석)

열차의 물리적 좌석으로, 여러 스케줄에서 공유

TrainSchedule

특정 날짜·노선의 운행 정보

ScheduleStop (정차역)

정차 순서 기반 구간 겹침 판단

Station(역)

역 마스터 데이터

기존 Booking 단일 엔티티의 문제

코레일처럼 좌석을 선택하면 일정 시간 동안 임시 점유(예약) 상태가 유지되고, 결제를 완료해야 확정 예매로 전환됩니다. 초기에는 이 두 상태를 Booking 단일 엔티티로 함께 관리했는데, 개발 과정에서 세 가지 구조적 문제가 드러났습니다.

문제 1 DB 락으로는 구간 충돌 제어 불가

Seat는 여러 TrainSchedule에서 재사용되는데, 비관적 락은 좌석 행 단위로 동작해 구간이 겹치지 않는 요청까지 불필요하게 대기시켰습니다.

문제 2 단일 책임 원칙(SRP) 위반

임시 점유와 확정 예매라는 두 책임을 하나의 엔티티가 담당해 상태 변경 시 관련 로직이 한 트랜잭션 안에 얹혔습니다.

문제 3 DB 스케줄러 기반 임시 예약 만료 처리의 한계

1분 주기 전체 스캔으로 실시간 만료가 불가능하고, 데이터가 많아질수록 I/O 부하가 증가했습니다.

개선 방향

개선 1 임시 예약(PendingBooking) 과 확정 예매(SeatBooking) 분리

임시 점유와 확정 예매의 책임과 생명주기를 명확히 분리했습니다.

개선 2 임시 예약을 Redis로 이전

TTL 기반 자동 만료로 스케줄러를 제거하고, 구간 충돌 검사를 Lua 스크립트로 원자적으로 처리하여 동시성 제어의 핵심 기반을 구축했습니다.

Redis 임시 예약 키 설계

동적 TTL - 출발 시간 기준 계산

왜 고정 10분이 아닌가?

출발 8분 전에 좌석을 임시 점유하고 결제를 하지 않으면, 고정 10분 TTL 적용 시 열차가 출발한 뒤에도 2분간 해당 예약이 목록에 남아 결제가 가능한 상태가 되는 문제가 발생합니다. 출발 시각까지 남은 시간을 기준으로 TTL을 계산해 반드시 출발 전에 임시 예약이 해제되도록 설계했습니다.

출발 30분 전
TTL = 10분

출발 8분 전
TTL = 8분

출발 5분 전
예약 마감

TTL = min (기본 10분, 출발까지 남은 시간)

출발 5분 전부터 예약 자체를 차단하는 검증 로직과 결합해 TTL 최솟값은 자연스럽게 5분이 됩니다.

예약 단위 개별 키 구조

하나의 키에 여러 예약을 담으면 새 예약이 추가될 때마다 TTL이 리셋됩니다. PendingBooking 마다 고유 키를 생성해 각각의 TTL을 독립적으로 관리할 수 있도록 했습니다.

Redis 키 구조 4종

여러 자료구조를 검토한 결과, 구간을 키가 아닌 멤버로 관리해 키 폭발을 방지할 수 있고, TTL 기반 자동 만료가 가능한 Set 구조를 채택했습니다.

키 패턴 / TTL	타입	Value 예시	용도
<code>{schedule:S}:seat:ID:hold:pbId</code> TTL: 동적	Set	<code>{"0-1", "1-2"}</code>	해당 예약이 점유한 구간 목록
<code>{schedule:S}:seat:ID:holds</code> TTL: 갱신형	Set	<code>{"pb_abc", "pb_def"}</code>	해당 좌석의 예약 ID 목록
<code>{pending-booking:{pbId}}</code> TTL: 동적	String	<code>PendingBooking{...}</code>	PendingBooking 객체 전체
<code>{pending-booking:member:{no}:{pbId}}</code> TTL: 동적	String	<code>"1"</code>	회원별 예약 조회 인덱스

구간 기반 좌석 공존 구조

정차역 0~7 구간에서 동일 좌석(42번)에 대해 두 예약이 발생한 상황입니다.

A는 0→3, B는 3→7 구간을 예약하며, 구간이 겹치지 않아 동일 좌석에 공존할 수 있습니다.

승객 A 예약 후 - 0→3 구간

Hold		
Key	Type	Value
<code>{schedule:1785}:seat:42:hold:PB_A</code>	Set	<code>{"0-1", "1-2", "2-3"}</code>
Holds		
Key	Type	Value
<code>{schedule:1785}:seat:42:holds</code>	Set	<code>{"PB_A"}</code>

승객 B 예약 후 - 3→7 구간 추가

Hold		
Key	Type	Value
<code>{schedule:1785}:seat:42:hold:PB_A</code>	Set	<code>{"0-1", "1-2", "2-3"}</code>
<code>{schedule:1785}:seat:42:hold:PB_B</code>	Set	<code>{"3-4", "4-5", "5-6", "6-7"}</code>
Holds		
Key	Type	Value
<code>{schedule:1785}:seat:42:holds</code>	Set	<code>{"PB_A", "PB_B"}</code>

예약 파이프라인 & 결제 도메인 설계

예약 → 결제 → 예매 파이프라인



왜 결제 승인 전 Order / Payment를 생성하는가?

“승인 이후 생성해도 되지 않을까?”라는 의문에서 출발했습니다. 하지만 Toss측 승인 이후 서버 장애가 발생하면, PG에는 결제 기록이 남지만 우리 DB에는 아무 기록도 없는 상태가 될 수 있습니다.

승인 후 생성

- 서버 장애 시 결제 고아 데이터 발생
- 복구, 정산, CS 대응이 어려움

승인 후 생성 (채택)

- 모든 결제 시도가 DB에 기록됨
- 장애 후에도 어느 단계에서 실패했는지 추적 가능

Payment를 ‘결제 결과’가 아닌 ‘결제 시도 자체’를 기록하는 이력으로 정의했습니다.

Order / Payment 분리 설계

Order (주문)

예약 정보와 기준 금액을 보관하며, 결제 요청 시 금액 검증 기준이 됩니다.

Payment (결제)

Toss와의 실제 결제 처리 정보를 관리하며, paymentKey와 승인/실패 이력을 저장합니다.

결제 기록과 비즈니스 계약이라는 서로 다른 책임을 분리했습니다.
엔티티 수를 줄이기보다 책임을 명확히 나누는 것이 장기적인 운영에 더 중요하다고 판단했습니다.

REQUIRES_NEW 트랜잭션 2단계 분리

paymentKey는 사용자가 Toss 결제 위젯에서 결제를 완료하면 발급되는 키입니다. 서버는 이 키로 Toss에 최종 승인을 요청하며, 승인 실패 시에도 결제 상태 조회·취소를 위해 반드시 보존되어야 합니다.

- Toss 호출 전 - paymentKey 즉시 커밋**

메인 트랜잭션과 분리된 트랜잭션으로 즉시 커밋합니다. 이후 메인 트랜잭션이 롤백되어도 paymentKey는 유지됩니다.
- Toss 호출 실패 - 실패 이력 즉시 커밋**

실패 사유와 에러 코드를 별도 트랜잭션으로 즉시 커밋합니다. 메인 트랜잭션 롤백과 무관하게 실패 이력이 보존됩니다.

구간 기반 동시성 제어 설계

동시성 제어 방식 선택

- 탈락

비관적 락 (Pessimistic Lock)
좌석 행 단위 잠금으로 인해 구간이 겹치지 않는 요청이나 다른 날짜 스케줄 예약까지 대기하는 과잉 잠금 문제가 발생했습니다. 또한 구간 조건을 원자적으로 처리하기 어려웠습니다
- 탈락

Redisson 분산 락
좌석 전체 단위로 락을 걸면 과잉 잠금이 발생하고, 구간 단위로 분할하면 부분 실패 시 보상 트랜잭션이 필요합니다. 네트워크 장애 시 보상 실패로 좌석이 잠긴 채 남을 수 있으며, 락 수 증가에 따라 데드락 위험도 함께 증가합니다.
- 채택

Redis Lua 스크립트
Redis 단일 스레드 모델과 Lua 스크립트를 활용해 락 없이 원자성을 보장하고, 좌석 점유를 구간 단위로 관리하여 요청 구간과의 교집합으로 충돌을 판단합니다.

Redis + RDB 이중 구조

	PendingBooking	Booking + SeatBooking
저장소	Redis	MySQL
역할	임시 좌석 점유	결제 완료 후 확정 예매
생명주기	TTL 기반 자동 만료 (최대 10분)	영구 보존
충돌 검사	Lua 기반 구간 충돌 검사 및 점유 처리	SQL 기반 구간 조건으로 최종 검증

Lua 스크립트 원자적 처리 구조

Redis 단일 스레드와 Lua 스크립트를 활용해 여러 단계 연산을 하나의 실행 단위로 묶어, 별도의 락 없이 원자적으로 처리하도록 설계했습니다.

- 1

Hold 목록 조회
해당 좌석에 임시 점유 중인 예약 ID 목록을 조회합니다.
- 2

Lazy Cleanup
TTL 만료로 삭제된 유령 데이터를 인덱스에서 즉시 제거하여 정합성을 유지합니다.
- 3

구간 겹침 검사
유효한 임시 예약의 점유 구간과 요청 구간의 교집합을 확인합니다.
구간이 하나라도 겹치는 경우 해당 좌석은 이미 점유된 상태로 판단하여 즉시 실패를 반환합니다.
- 4

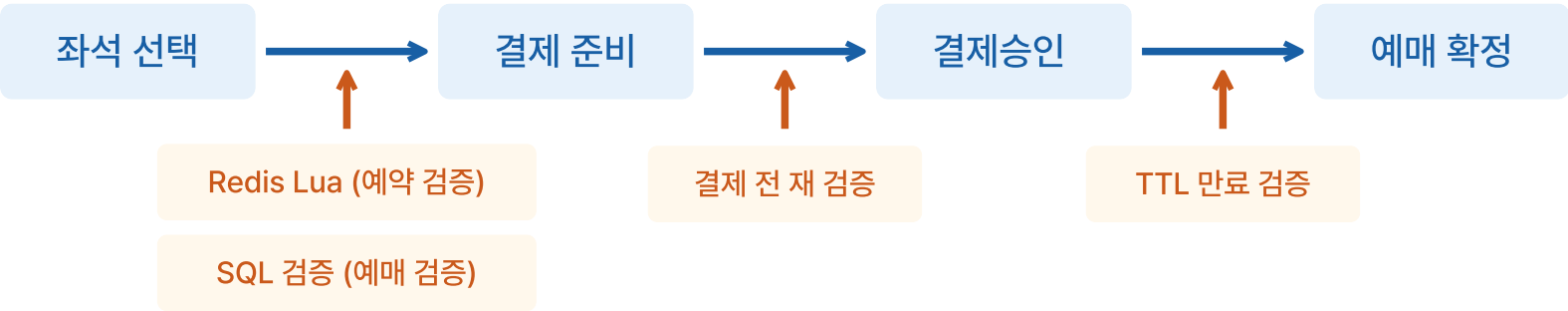
신규 점유 등록 및 인덱스 업데이트
충돌이 없는 경우, 요청 구간을 신규 점유로 등록하고 관련 인덱스를 함께 업데이트합니다.



락 기반 제어의 한계를 해결하기 위해 상태를 원자적으로 변경하는 방식으로 전환하여, 구간 기반 동시성 문제를 구조적으로 해결했습니다.

좌석 충돌 방지를 위한 4단계 검증 구조

좌석 충돌 판단은 단일 검증으로 해결되지 않습니다.
좌석 선택부터 결제 승인까지 각 단계에서 서로 다른 유형의 충돌을 방어하도록 4단계 검증 구조를 적용했습니다.



- 1

Redis Lua 스크립트

예약 vs 예약 — 1차 차단

좌석 선택 시점에 임시 예약(Hold) 간 구간 겹침을 원자적으로 검사합니다.
동시 요청 상황에서도 겹치는 구간의 중복 선점을 즉시 차단합니다.
- 2

SQL 구간 겹침 조건

예약 vs 확정 예매 - 교차 검증

Redis Hold 선점 이후, DB의 확정 예매와 구간 충돌 여부를 SQL로 검증합니다.
이미 결제가 완료된 좌석 구간과의 충돌을 차단합니다.
- 3

결제 준비 재검증

방어적 중복 검증

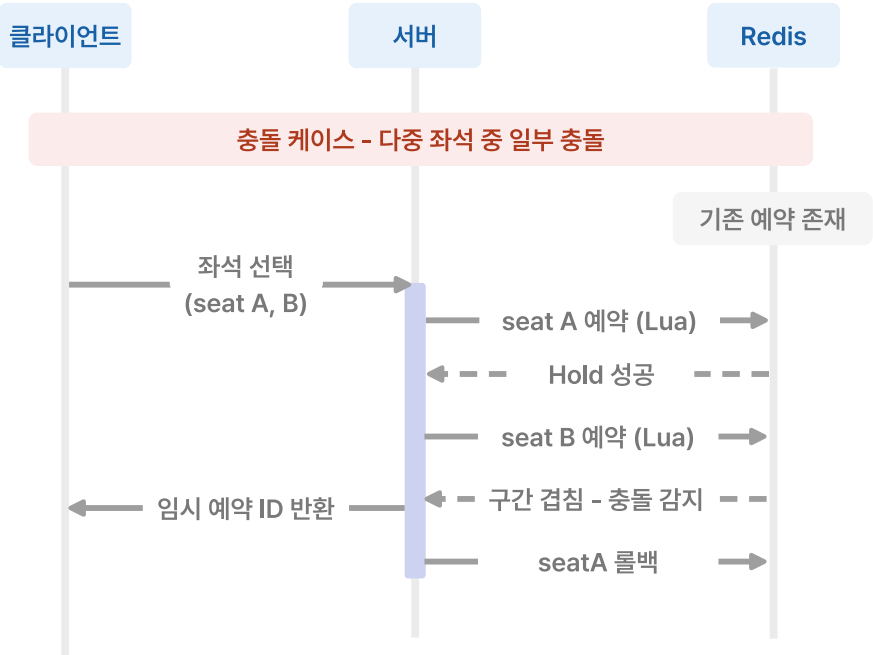
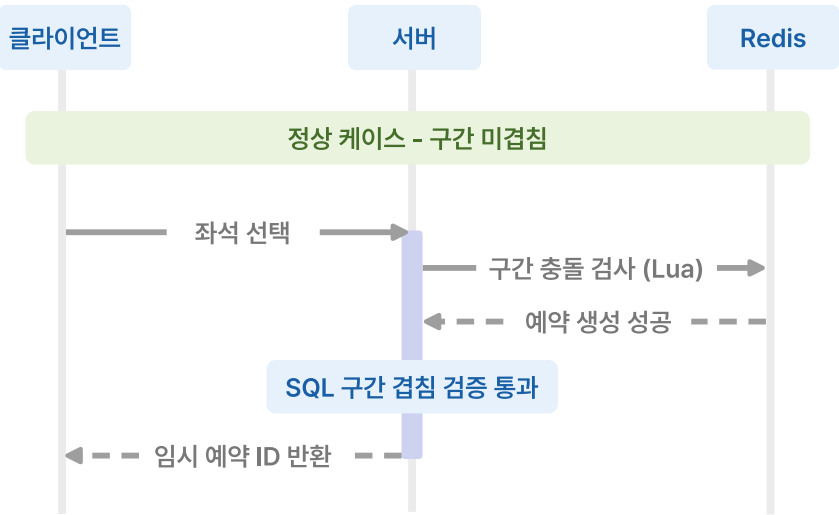
결제 준비 시점에 확정 예매와의 충돌 여부를 한 번 더 확인합니다.
대부분 이전 단계에서 차단되지만, TTL 타이밍 등 예외 상황을 대비한 방어적 검증 레이어로, 검증 비용이 낮아 추가 안전장치로 적용했습니다.
- 4

TTL 기반 검증

만료 예약 결제 차단

결제 승인 시점에 Redis에 있는 임시 예약의 유효성을 확인합니다.
TTL이 만료된 임시 예약으로 결제가 진행되는 것을 최종 차단합니다.

정상 케이스 vs 충돌 케이스 시퀀스



보상 트랜잭션 & 롤백 안정성

4단계 검증 구조 위에 보상 트랜잭션을 결합하여, 다중 좌석 선점 중 일부 실패 시에도 전체 상태 일관성을 유지하도록 구현했습니다.

다중 좌석 선점 중 일부 실패 시나리오

- 성공** 좌석 A, B 선점 완료
- 충돌** 좌석 C 선점 실패 → 보상 트랜잭션 실행
- 복구** seat_release.lua로 A, B 선점 원자적 해제

Toss Payments 연동 - 결제 정합성 보장 구조

금액 4단계 교차 검증

외부 결제 API와 연동하는 과정에서 결제 금액 위변조와 중복 결제를 차단하기 위해 클라이언트부터 Toss 응답까지 4단계로 교차 검증합니다.

- 1

클라이언트 ↔ Order

사용자가 전달한 결제 금액과 Order의 기준 금액을 비교하여 잘못된 요청 금액을 차단합니다.
- 2

Order ↔ Payment

결제 준비 단계에서 생성된 Payment 금액과 Order 금액을 비교하여 내부 데이터 불일치를 방지합니다.
- 3

paymentKey 검증

클라이언트가 전달한 paymentKey와 DB에 저장된 값을 비교하여 중복 결제 및 위조 요청을 차단합니다.
- 4

Toss 응답 ↔ Order

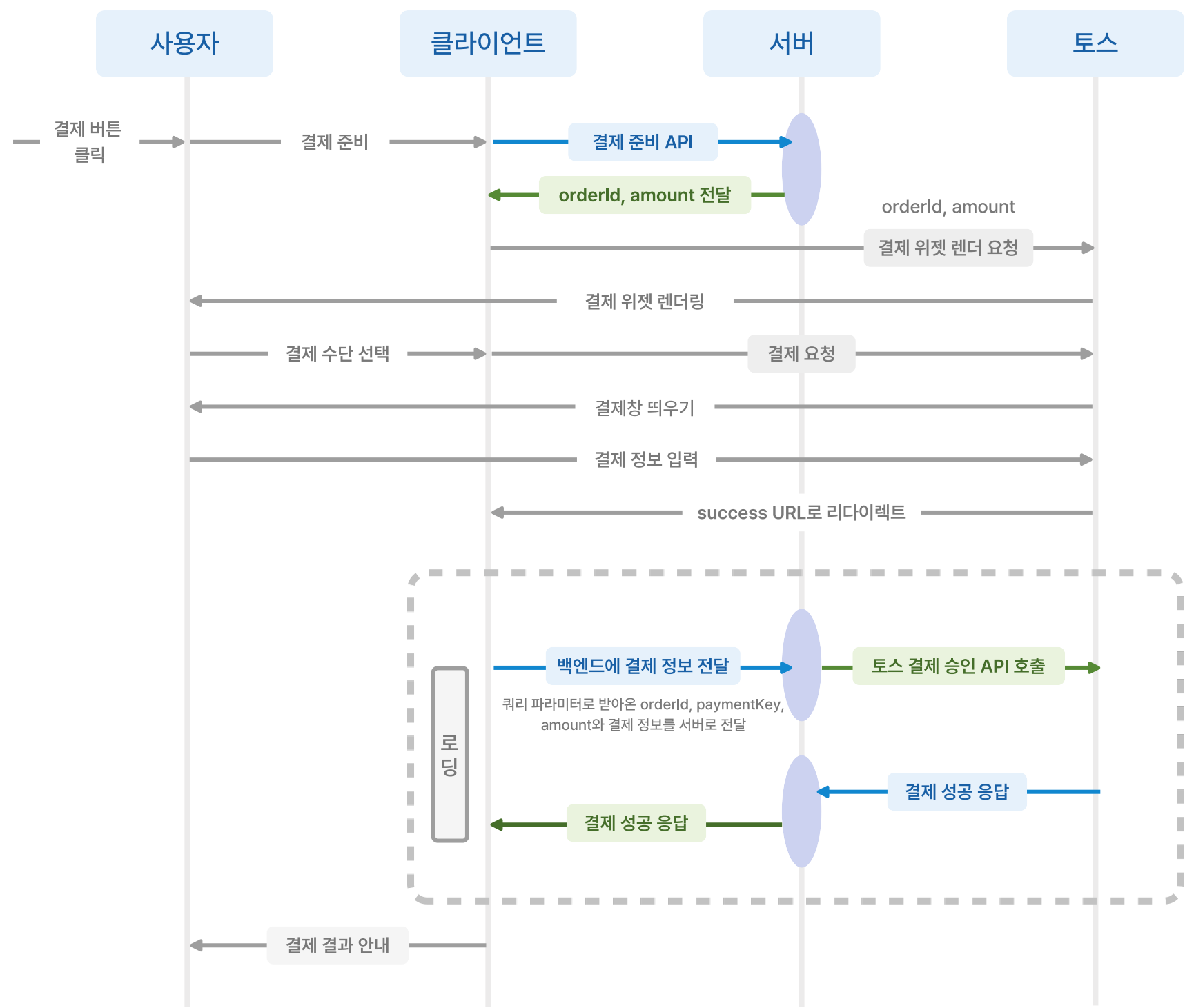
Toss API 승인 응답 금액과 Order 금액을 최종 비교하여 외부 응답 기준 금액 변조를 방지합니다.

HTTP 클라이언트 선택 & 에러 처리 구조

RestClient + retrieve() + onStatus() 채택

JPA 기반 동기 애플리케이션 구조와 일치하도록 Spring Boot 3.2+의 RestClient를 선택했습니다.
HTTP 상태 코드별 에러를 명확히 처리하기 위해 retrieve() + onStatus() 조합으로 4xx/5xx 응답 시 body를 파싱해 TossPaymentException으로 변환하는 구조를 적용했습니다.

결제 시퀀스 다이어그램



트러블 슈팅 - Toss API HTTP 500 빈 응답 처리

증상

결제 승인 API 호출 시 HTTP 500 + 빈 body가 반환되어, 실패 응답의 에러 코드를 파싱하고 Payment 이력으로 남기는 것이 불가능했습니다.

버그 메커니즘

빈 body로 인해 Jackson 파싱 예외가 발생했고, Spring 내부에서 처리되면서 onStatus()로 전달되지 않았습니다.
→ 결과적으로 TossPaymentException이 발생하지 않았습니다.

원인 파악 과정

- 오판

응답 문제로 접근
retrieve() + onStatus() 조합을 의심해 여러 방법을 시도했지만 증상이 동일했습니다.
- 전환

요청 확인
requestInterceptor로 헤더를 로깅하여 확인한 결과 .env 미설정으로 인해 \${TOSS_SECRET_KEY}:가 그대로 전송되고 있음을 확인했습니다.
- 검증

curl 재현

\${ } 가 포함 credential	HTTP 500 + 빈 body - 응답 헤더에서 Envoy 확인
일반적인 잘못된 credential	HTTP 401 + 정상 JSON body

→ 동일한 인증 실패라도 요청 값에 따라 응답 형태가 달라졌고, 특정 요청 패턴에서 인증 처리 경로가 다르게 동작하는 케이스로 판단했습니다.

해결

- 해결 1

retrieve() + onStatus() → exchange() 전환
retrieve()는 응답 처리가 추상화되어 있어 예외 흐름을 세밀하게 제어하기 어려웠습니다.
exchange()로 전환해 응답 전체에 직접 접근하면서, 빈 body 및 파싱 실패를 TossPaymentException으로 명시적으로 처리했습니다.

→ 해당 메커니즘은 재현 테스트로 확인했습니다.
- 해결 2

설정값 검증 강화
Toss API 설정값을 @ConfigurationProperties로 관리하고 있었는데, 미해석 플레이스홀더가 그대로 바인딩되는 특성 때문에 \${TOSS_SECRET_KEY}가 실제 값으로 치환되지 않은 채 런타임까지 전달되는 문제가 있었습니다.

@Value로 전환하는 방안도 검토했지만, 설정값을 객체 단위로 관리하면서 타입 안전성과 테스트 용이성을 유지하기 위해 @ConfigurationProperties + record 구조를 유지하기로 결정했습니다.

대신 @Validated와 생성자 검증을 추가해 .env 미설정 시 부팅 시점에 즉시 실패하도록 보완했습니다.

배운 점

이러한 유형의 문제를 분석할 때는 응답뿐 아니라 실제 전송된 요청까지 함께 확인해야 한다는 점을 배웠습니다.
또한 환경변수와 같은 외부 설정값은 반드시 부팅 시점에 검증해, 잘못된 값이 런타임까지 전파되지 않도록 해야 합니다.

예약 좌석 가용성 계산 개선 - 객차 단위 Hold 인덱스 설계

문제 배경

열차 조회 API는 carType별, 객차 조회API는 객차별 잔여 좌석 계산이 필요했습니다.

하지만 기존 구조에서는 임시 예약(Hold)이 좌석 단위 키에 분산 저장되어 있어, 좌석마다 Hold를 개별 조회해야했고 페이징 1회(열차 10대 기준) 최소 9,600번의 Redis 조회가 발생했습니다.
또한 구간 겹침 판단까지 필요해 단순 카운트 방식으로는 해결할 수 없었습니다.

설계 방향

기존 per-seat 구조는 충돌 검사용으로 유지하되, 가용성 계산 경로만 조회 API 단위에 맞춰 최적화하기로 했습니다.
→ 전체 좌석을 순회하던 조회 방식 대신, 필요한 데이터만 조회할 수 있도록 인덱스를 추가했습니다.

자료구조 검토

방식	채택	판단 근거
전체 좌석 Pipeline 조회	탈락	싱글 스레드 특성상 수만 건 명령이 큐에 적체되어 전체 응답 시간에 영향을 줍니다.
구간별 카운터	탈락	같은 좌석이 여러 구간에 Hold되면 1석이 중복 카운트 되고, TTL 만료 시 감소가 보장되지 않아 오차가 발생합니다.
Bitmap	탈락	Bitmap은 비트 단위 TTL이 불가능해 별도 정리 로직이 필요하고, 노선 변경 시 오프셋 재설계가 필요합니다.
Hold 좌석 인덱스 (Sorted Set)	채택	기존 구조를 유지하면서 정확도와 조회 성능을 동시에 확보할 수 있습니다.

Sorted Set 선택 이유

Hold 만료 시각을 score로 저장하면 ZRANGEBYSCORE (now ~ +inf) 조회만으로 유효한 Hold를 즉시 필터링할 수 있습니다.

→ 별도 정리 로직 없이 기존 TTL + Lazy Cleanup 흐름을 그대로 활용할 수 있습니다.

최종 키 구조

Key	{schedule:1785}:traincar:231:holding-seats	
Type	Sorted Set	
Member	seatId:section	예: "42:0-1", "42:1-2"
Score	https://github.com/penggu-dev/raillo-batch	

그 결과, Redis 명령어 수를 9,600건 → 180건 수준으로 크게 줄일 수 있었습니다.
(열차 10대 × 객차 최대 18개)

열차 조회 API 최적화

문제 배경

열차 조회는 Raillo 서비스에서 가장 빈번하게 호출되는 핵심 기능입니다. K6 부하 테스트 결과 해당 API에서 응답 시간 병목이 확인되었고, 원인 분석을 통해 외래키 인덱스 비효율과 열차별 좌석·예매 정보를 개별 조회하는 구조를 주요 원인으로 식별했습니다. 이에 따라 쿼리 구조 개선, 인덱스 재설계, 조회 방식 최적화의 세 가지 방향으로 성능 개선을 진행했습니다.

2단계 쿼리 분리

서브 쿼리	후보 열차 선필터 출발역과 도착역을 모두 경유하면서, 출발역이 도착역보다 먼저 정차하는 열차만 추려냅니다. (예: 서울→부산 검색 시 부산→서울 방향 열차 제외) 이 결과를 메인 쿼리의 조건으로 사용합니다.
메인 쿼리	열차 상세 정보 조회 서브쿼리로 걸러낸 열차들을 대상으로 열차 정보, 출발/도착 시각, 역 정보를 한 번에 조회합니다.

Covering Index 재설계 - 단독 FK 인덱스 제거 및 ScheduleStop 중심 조회 최적화

활용도가 낮은 단독 FK 인덱스와 복합 인덱스가 혼재하던 구조를, 실제 조회 패턴에 맞는 2개의 복합 Covering Index 로 재설계해 인덱스 수를 줄이고 추가 테이블 접근 없이 필터링이 가능하도록 최적화했습니다.

인덱스	구성 컬럼	효과
idx_stop_depart_filter	출발역 ID, 출발시간, 스케줄 ID, 정차 순서	Covering range scan - 테이블 추가 접근 없음
idx_stop_arrival_filter	도착역 ID, 스케줄 ID, 정차 순서	ICP(인덱스 조건 푸시다운)로 방향 비교를 인덱스 레벨 처리

배치 조회 도입 - 열차별 개별 조회 → IN절 일괄 조회

열차 10대의 좌석·예매 정보를 각각 조회하던 구조를 한 번의 쿼리로 전환했습니다.

findTrainSeatInfoBatch	좌석 등급(일반실·특실)별 전체 좌석 수를 배치 조회
findOverlappingBookingsBatch	검색 구간과 겹치는 확정 예매를 배치 조회
getTrainCarIdsBatch	검색된 열차 스케줄의 객차 ID를 배치 조회

운영 캘린더 Redis 캐시 적용

한 달치 운영 가능 날짜는 배치로 생성되며 변경이 적습니다. 매 요청마다 DB를 조회하던 구조를 Redis TTL 캐시로 전환해 불필요한 반복 조회를 제거했습니다.

개선 결과

65% ↓ 응답시간 개선	87% ↑ 처리량 향상	3.64ms 메인 쿼리 실행시간	275ms 전체 응답시간
------------------	-----------------	----------------------	------------------

RDS t3.micro 환경에서 약 2개월치 데이터(약 20,000건의 열차 스케줄)를 기준으로 성능을 측정했습니다

커넥션 풀 & 스레드 풀 튜닝

문제 발견 및 원인 분석

부하 테스트는 EKS 환경(1 vCPU / 1Gi / 2 replicas / t3.small)에서 수행하여 동시 사용자 0→300 VU(5분 ramp-up) 조건과 검색 60% / 객차 조회 10% / 좌석 조회 30% 비율로 부하를 발생시켰습니다. 또한 약 2개월치 열차 스케줄 (약 20,000건)과 예매 데이터 200,000건 규모의 데이터 환경에서 테스트를 진행했습니다.

- 1

고부하 환경에서 응답 지연 지속

쿼리 최적화 이후에도 VU 300 조건에서 p90이 3.87s로 여전히 높았습니다.
- 2

쿼리 병목 여부 검증

performance_schema로 분석한 결과, 슬로우 쿼리는 발견되지 않았습니다.
→ DB 쿼리는 병목이 아니라고 판단했습니다.
- 3

커넥션 풀 및 스레드 풀 조합 검증

커넥션 풀(HikariCP)과 Tomcat 스레드 풀 설정을 조합해 VU 300 / 5분 조건으로 5차 실험을 진행했습니다.

실험 결과 및 최적 설정

차수	스레드	커넥션	RPS	p90
1차	200	10	97/s	3.87s
2차	20	20	107/s	3.35s
3차	25	25	115/s	2.69s
4차	30	25	112/s	2.64s
5차	30	30	106/s	3.0s

종합 성능(RPS + p90)을 기준으로 25/25 설정을 최적 구성으로 채택했습니다.

최적 설정 도출 과정

- 1

과도한 스레드의 오버헤드

스레드를 200 → 20으로 줄이자 RPS가 증가했습니다.
→ 과도한 스레드가 컨텍스트 스위칭 비용을 유발하고 있었습니다.
- 2

스레드와 커넥션의 균형

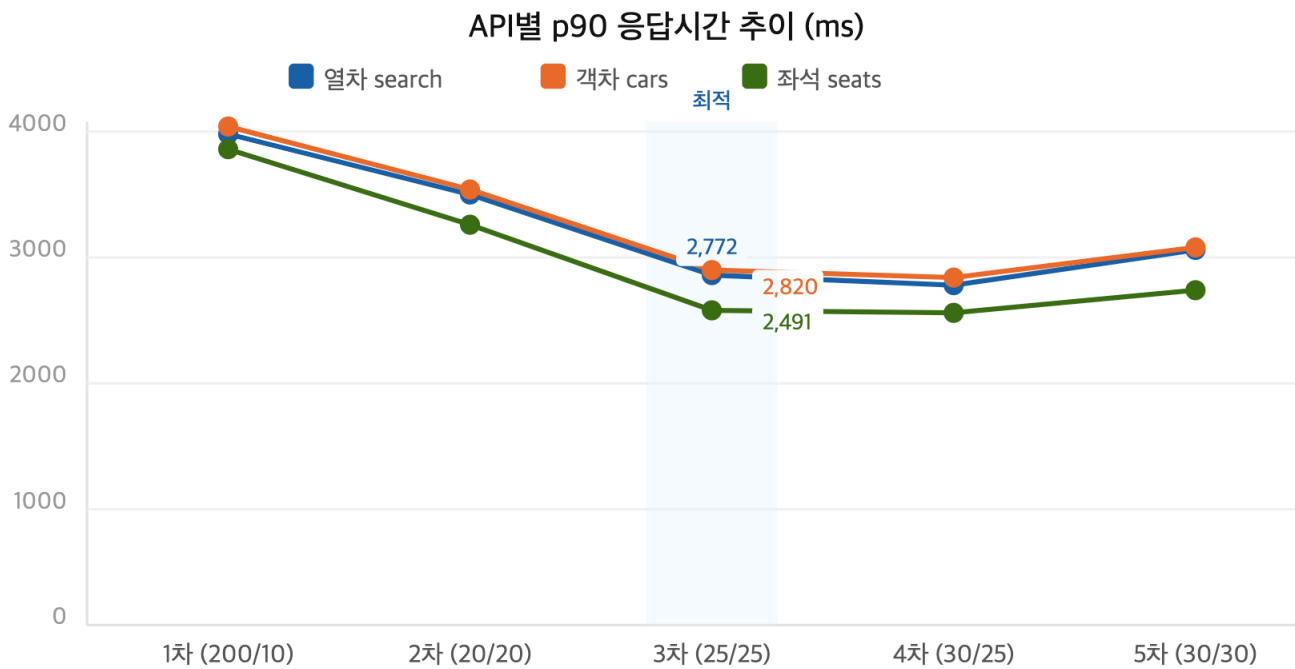
25/25 설정에서 RPS와 p90이 가장 안정적인 결과를 보였습니다.
→ I/O bound 환경에서는 스레드 수와 DB 커넥션 수가 균형을 이룰 때 처리 효율이 가장 높았습니다.
- 3

스레드 증가의 한계

스레드를 30으로 늘리자 p90은 소폭 개선되었지만 RPS가 감소했습니다.
→ DB 커넥션 수 대비 스레드가 많아지면서, 커넥션을 얻지 못한 스레드들이 대기 상태에 머물러 전체 처리량이 감소했습니다.
- 4

커넥션 증가에 따른 DB 부하

커넥션을 30까지 늘리자 RPS와 p90 모두 악화되었습니다.
→ 동시 DB 요청 수가 증가하며 DB 부하로 이어진 것으로 판단했습니다.



AOP 기반 메트릭 수집 & Grafana 관측성

문제 배경

인프라 메트릭(CPU, 메모리, 요청 수)만으로는 “결제가 왜 실패했는지”, “좌석 충돌이 얼마나 발생하는지”를 파악하기 어렵습니다.

예약·결제 도메인 특성을 반영한 비즈니스 메트릭을 수집하기 위해, AOP 기반 메트릭 수집 구조를 구현했습니다.

- 예약

임시 예약 생성 처리량 → 피크 시간대 부하를 정량적으로 확인
- 충돌

임시 예약 충돌 / 확정 예매 충돌 빈도 → 동시성 제어 설계의 효과 검증
- 결제

결제 성공률 · 실패 원인 · Toss API 응답 시간 → 외부 API 장애와 비즈니스 오류 구분

결제 도메인 Aspect 메트릭 구조

Aspect	메트릭	타입	설명
PaymentMetrics	payment_prepare_total	Counter	결제 준비 호출 수
	payment_confirm_success_total	Counter	결제 승인 성공 수
	payment_confirm_failure_total	Counter	승인 실패 수 (reason, error_code 태깅)
TossApiTimer	toss_api_duration_seconds	Histogram	Toss API 응답 시간 (operation 태깅)

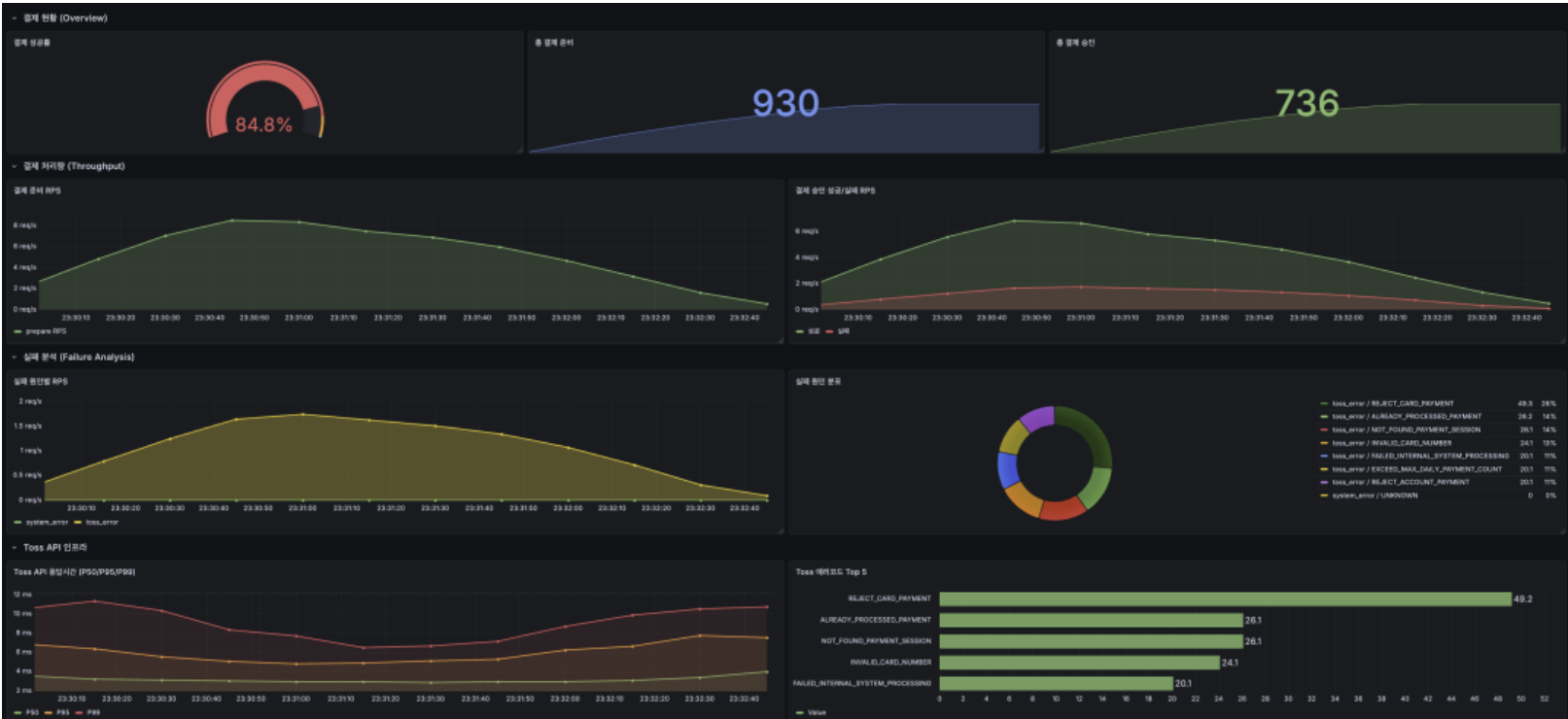
실패 이중 태깅 - reason + error_code

reason(toss_error / validation_error / system_error)과 error_code(REJECT_CARD_PAYMENT 등)를 함께 태깅해 외부 API 장애 / 비즈니스 검증 실패 / 시스템 오류를 한눈에 구분할 수 있도록 했습니다.

Payment Grafana 대시보드

대시보드 JSON을 Git으로 관리하고, 로컬 환경에서는 Docker Compose 기반 자동 프로비저닝을 적용했습니다.

이미 ArgoCD 기반 GitOps 구조가 구성되어 있어, 운영 환경에도 동일한 방식으로 확장 가능하도록 설계했습니다.



이미지 링크:

<https://github.com/user-attachments/assets/d32e5148-1b7d-4c1e-b6c8-f74122782a8c>

테스트 인프라 & 결제 테스트 전략

문제 인식

결제는 외부 API(Toss)에 의존하기 때문에, 실제 장애 상황이나 실패 케이스를 안전하게 재현하기 어렵습니다.
또한 단순 단위 테스트만으로는 실제 HTTP 통신과 메트릭 흐름을 검증하기 어려웠습니다.

외부 연동 API 테스트 전략

검증 목적에 따라 테스트 도구를 분리하고, 하나의 테스트에 모든 책임을 두기보다 역할별로 나누어 구성했습니다.

도구	목적	적용 대상
Mockito	Redis	Service / Facade
@RestClientTest	HTTP 클라이언트 단위 검증	TossPaymentClient
WireMock	실제 HTTP 시나리오 검증	Toss API 연동

테스트 환경 구성

Raillo에서는 로컬 테스트를 위해 Docker Compose로 서버·Redis·WireMock·Prometheus·Grafana를 함께 구성하고 있습니다.
해당 환경에서 Toss API 연동을 WireMock으로 교체해 외부 API 없이 결제 플로우 전체를 재현했습니다.



WireMock 시나리오 설계

Toss Payments 공식 에러 문서를 참고하여 실제 에러 코드·HTTP 상태·메시지를 그대로 재현했습니다.
paymentKey에 에러 코드를 포함시키면 WireMock이 패턴 매칭으로 해당 실패 응답을 반환하는 구조입니다.

- 성공

ERR_ 미포함 paymentKey → WireMock 200 응답 → 정상 결제
- 실패

ERR_ 포함 paymentKey (ex) ERR_REJECT_CARD_PAYMENT)→ 4xx 에러 응답 반환

메트릭 검증 - K6 부하 테스트 연계

VU 30 기준 임시 예약 → 결제 준비 → 결제 승인 플로우를 반복 실행합니다.
80% 성공 / 20% 실패 시나리오로 메트릭 수집이 실제로 동작하는지 검증했습니다.

검증 결과

- 외부 API 없이 결제 플로우를 재현할 수 있었습니다.
- 성공 / 실패 케이스를 분리해 결제 및 예외 처리 흐름을 검증했습니다.
- 메트릭 수집과 Grafana 시각화까지 정상 동작을 확인했습니다.

클라우드 인프라 & 배치 자동화

네트워크 구성 - 2-AZ, Public / Private 서브넷 분리

외부 요청은 IGW를 통해 유입된 뒤 ALB를 거쳐 Private 서브넷의 서비스로 전달되도록 구성했습니다.

PUBLIC 서브넷 (2-AZ)
IGW, NAT Gateway, ALB
PRIVATE 서브넷 (2-AZ) - 외부 직접 접근 차단
EKS 노드, RDS (MySQL), ElastiCache (Redis)

Kubernetes 배포 전략

분산 배포	topologySpreadConstraints Raillo Spring 애플리케이션에 해당 설정을 적용해 노드 및 AZ 기준으로 Pod를 균등하게 분산했습니다. 이를 통해 특정 AZ 장애 발생 시에도 서비스가 유지되도록 구성했습니다.
무중단	readinessProbe + replicas 2 준비된 Pod에만 트래픽을 전달하도록 구성했습니다. 롤링 업데이트 과정에서도 기존 Pod가 유지되기 때문에 요청 손실 없이 배포가 이루어집니다.

배치 서버 분리 - railo-batch

열차 데이터 적재는 사용자 요청과는 성격이 다른 관리자 도메인 작업입니다. 배포 주기·리소스·장애 영향 범위를 독립적으로 관리해 실시간 서비스와 배치가 서로 영향을 주지 않도록 별도 서버로 분리했습니다.

BATCH_MODE 기반 실행 제어

하나의 배치 애플리케이션에서 BATCH_MODE 값에 따라 엑셀 파싱, 하루 단위 스케줄 생성, 한 달 단위 초기 적재를 분기 실행하도록 구성했습니다.

parse	엑셀 파싱 - 스케줄 템플릿 + 운임 데이터 적재
day	다음 운행일 하루치 스케줄 생성. BATCH_DATE 지정 시 해당 날짜로 실행
month	오늘부터 한 달치 스케줄 생성
init	parse + month 순차 실행 — 최초 세팅용 (DDL create)

환경별 배치 실행 전략

배치는 환경에 따라 서로 다른 방식으로 실행되도록 구성했습니다.

개발	GitHub Actions의 스케줄 기능을 사용해 매일 KST 02:00 에 자동 실행되도록 설정했습니다. 또한 workflow_dispatch를 통해 init / day 모드를 수동으로 실행할 수 있도록 구성했습니다.
운영	Kubernetes CronJob을 사용해 매일 KST 02:00 에 자동 실행되도록 구성했습니다. 이에 concurrencyPolicy: Forbid 설정을 통해 동일 배치의 중복 실행을 방지했습니다.

BatchRunner - 배치 실행 제어

BatchRunner는 Spring Boot의 CommandLineRunner를 구현해 애플리케이션 시작 시 즉시 배치가 실행되도록 구성했습니다.

작업 완료 후 System.exit(0)으로 프로세스를 명시적으로 종료해 Kubernetes CronJob이 Job 완료를 정상적으로 인식하고, 실패 시에는 System.exit(1)로 종료해 backoffLimit 기반 재시도가 가능하도록 구성했습니다.